

FX Risk

Guide du projet pour debutants

De zero a la comprehension complete

Langage simple - Organisation des fichiers - Installation pas a pas

Ce document explique le projet FX Risk avec un langage simple. Il est concu pour une personne qui decouvre le developpement web, React, Supabase ou la gestion des risques de change. Aucune connaissance prealable n'est requise.

1. Qu'est-ce que ce projet ?

FX Risk est une application web qui aide une banque ou une équipe financière à suivre les opérations de change (Foreign Exchange, abrégé FX). Quand quelqu'un achète ou vend des devises (euro, dollar, etc.), l'application enregistre l'opération, calcule automatiquement son niveau de risque, génère des alertes si quelque chose est suspect, et permet aux différents services de valider ou rejeter la transaction.

En résumé : c'est un tableau de bord intelligent pour surveiller les risques liés aux opérations FX, avec des rôles utilisateurs (Front Office, Back Office, Risk Team, Manager, Admin).

1.1 Le problème résolu

- Centraliser toutes les opérations FX au même endroit.
- Calculer automatiquement un score de risque de 0 à 100.
- Alerter en cas d'anomalie : montant élevé, SWIFT manquant, horaire atypique.
- Faire valider les opérations par les bonnes équipes.
- Garder une trace de qui a fait quoi (journal d'audit).
- Afficher des indicateurs (KRI), graphiques et prédictions.

1.2 Exemple concret

Un opérateur Front Office saisit : Client ABC, achat de 2 000 000 USD contre EUR, sans référence SWIFT, à 22h. Le système calcule un score élevé, crée une alerte « SWIFT manquant », et place l'opération dans la file de validation. Le Back Office voit la recommandation « Demander confirmation SWIFT » et peut valider ou rejeter.

2. Technologies utilisées (expliquées simplement)

Le projet repose sur plusieurs briques. Voici ce que chacune fait, en langage courant :

React 19

Bibliothèque JavaScript pour construire l'interface : boutons, tableaux, graphiques. C'est ce que l'utilisateur voit à l'écran.

TypeScript

JavaScript avec des types. Cela aide à éviter les erreurs avant d'exécuter le code (par exemple : un montant doit être un nombre).

Vite

Outil qui lance le projet en développement et prépare la version finale. Commande principale : `npm run dev`.

TanStack Router / Start

Gère les pages (`/login`, `/app`, `/app/operations...`) et le rendu côté serveur (SSR). Chaque URL correspond à un fichier dans `src/routes/`.

Supabase

Backend dans le cloud : base de données PostgreSQL, authentification (login), sécurité par rôles (RLS), mises à jour en temps réel. Des triggers SQL calculent le risque et créent les alertes automatiquement à chaque insertion.

Tailwind CSS + shadcn/ui

Styles et composants prêts à l'emploi (cartes, boutons, formulaires) pour une interface moderne et cohérente.

Recharts

Bibliothèque de graphiques (courbes, camemberts, barres) utilisée sur le tableau de bord.

jsPDF

Génère les rapports PDF exportables depuis la page Rapports de l'application.

3. Organisation des fichiers

Pour un débutant, retenir trois zones : `src/` (code de l'application), `supabase/` (base de données), racine (configuration).

3.1 Racine du projet

FX-Risk/	
package.json	Dépendances npm et scripts (dev, build, lint)
vite.config.ts	Configuration du build Vite
tsconfig.json	Configuration TypeScript
.env.example	Modèle des variables d'environnement
README.md	Documentation principale du projet
docs/	Documentation générée (ce PDF)

3.2 Dossier `src/` - cœur de l'application

src/	
routes/	Une page web = un fichier ici
components/	Blocs réutilisables (menu, badges, dialogues)
lib/	Logique métier (auth, risque, PDF, utilitaires)
hooks/	Hooks React (ex: rafraîchissement temps réel)
integrations/	Connexion Supabase (client, auth, types)
router.tsx	Point d'entrée du routeur
server.ts	Serveur SSR
styles.css	Styles globaux Tailwind

3.3 Dossier `src/routes/` - les pages web

Convention TanStack Router : le nom du fichier devient l'URL.

- `login.tsx` Page de connexion
- `app.index.tsx` Tableau de bord (KRI, graphiques)
- `app.operations.index.tsx` Liste des opérations FX
- `app.operations.new.tsx` Créer une nouvelle opération
- `app.validation.tsx` File de validation
- `app.approvals.tsx` File d'approbation (manager)
- `app.alerts.tsx` Centre d'alertes
- `app.heatmap.tsx` Cartographie des risques
- `app.predictions.tsx` Prédiction des risques (7 jours)
- `app.reports.tsx` Rapports PDF et CSV
- `app.audit.tsx` Journal d'audit

- app.clients.index.tsx Annuaire clients
- app.clients.\$clientId.tsx Fiche detaillee d'un client
- app.admin.users.tsx Gestion des utilisateurs
- app.admin.settings.tsx Parametres admin

3.4 Dossier src/components/

- app-shell.tsx : barre laterale, menu de navigation selon le role.
- operation-detail-dialog.tsx : fenetre de detail d'une operation.
- risk-indicators.tsx : badges de niveau de risque (low, high, etc.).
- ui/ : composants shadcn (Button, Card, Table, Dialog...).

3.5 Dossier src/lib/ - logique importante

- auth.ts : session utilisateur, roles, permissions.
- risk.ts : formatage montants, recommandations, helpers risque.
- pdf.ts : export PDF des rapports operations/alertes.
- utils.ts : fonctions utilitaires communes.

3.6 Dossier supabase/

```
supabase/
  migrations/           Scripts SQL (tables, regles, triggers, policies)
  seed_operations.sql   Donnees de test (optionnel)
  config.toml           Configuration Supabase locale
  SEED_DATA.md          Instructions pour les donnees de demo
```

Les migrations creent notamment : profiles, user_roles, operations, alerts, audit_logs, et les fonctions compute_risk() et generate_alerts() executees automatiquement.

4. Flux complet de l'application

- 1. Connexion sur login.tsx via Supabase Auth.
- 2. Lecture du role (front_office, back_office...) via lib/auth.ts.
- 3. Le menu (app-shell.tsx) affiche uniquement les pages autorisees.
- 4. Front Office cree une operation : insertion dans la table operations.
- 5. Trigger SQL compute_risk() : calcule risk_score (0-100) et risk_level.
- 6. Trigger generate_alerts() : cree des alertes si anomalies detectees.
- 7. Operation visible dans File de validation (statut : pending).
- 8. Back Office / Risk / Manager / Admin : Valider ou Rejeter.
- 9. Dashboard, alertes et graphiques se mettent a jour (temps reel).
- 10. Actions enregistrees dans audit_logs pour la tracabilite.

5. Les cinq roles utilisateurs

RBAC = controle d'accès basé sur les rôles. Chaque utilisateur a un seul rôle.

- front_office : cree des operations, consulte clients et historique.

- back_office : valide ou rejette les operations en attente.
- risk_team : supervise alertes, cartographie, validation, predictions.
- manager : approuve les cas critiques ou escalades.
- admin : acces complet, gestion utilisateurs et parametres.

6. Modules fonctionnels en detail

Tableau de bord (app.index.tsx)

Affiche les KRI : total operations, operations a haut risque, alertes ouvertes, score moyen, operations en attente, exposition. Graphiques : tendance 14J/30J/12M, distribution des risques, paires de devises.

Operations FX

Historique complet : reference, client, contrepartie, montant, taux, statut, score. Creation via formulaire guide.

File de validation

Operations en attente avec recherche, score, recommandations automatiques (SWIFT, montant, horaire...) et actions Valider / Rejeter.

Centre d'alertes

Alertes generees automatiquement : severite, categorie, message. Possibilite d'acquitter.

Cartographie des risques

Matrice Probabilite x Impact calculee dynamiquement a partir des operations courantes.

Prediction des risques

Projection heuristique sur 7 jours : risque moyen prevu, tendance, volume haut risque, exposition prevue.

Rapports et audit

Export PDF/CSV des operations et alertes. Journal d'audit pour tracer les actions sensibles.

7. Scoring et alertes automatiques

Le score (0-100) prend en compte : montant eleve, SWIFT manquant ou invalide, horaire atypique, ecart de taux, paire inhabituelle.

Classification : low (≤ 20), moderate (≤ 40), high (≤ 60), critical (> 60).

Types d'alertes : High Amount, Missing SWIFT, Late-Night Operation, Critical Risk Score.

8. Installation depuis zero

Prerequis

- Node.js installe (<https://nodejs.org>) - version LTS recommandee.
- Compte Supabase gratuit (<https://supabase.com>).
- Editeur de code : VS Code ou Cursor.

Etapes pas a pas

- Etape 1 : Ouvrir le dossier du projet dans un terminal.

```
cd FX-Risk
npm install
```

- Etape 2 : Copier .env.example vers .env et remplir les clés Supabase.
- Etape 3 : Dans Supabase > SQL Editor, exécuter les migrations dans l'ordre chronologique.
- Etape 4 : Désactiver la confirmation email (Auth > Settings) pour les comptes demo.
- Etape 5 : Lancer l'application.

```
npm run dev
```

- Etape 6 : Ouvrir http://localhost:5173 dans le navigateur.

Variables d'environnement (.env)

- VITE_SUPABASE_URL : URL de votre projet Supabase.
- VITE_SUPABASE_PUBLISHABLE_KEY : clé publique (anon key).
- SUPABASE_SERVICE_ROLE_KEY : clé serveur (ne jamais exposer côté client).

9. Comptes de démonstration

Mot de passe pour tous les comptes : Demo!2026

- front@fxrisk.demo - Front Office
- back@fxrisk.demo - Back Office
- risk@fxrisk.demo - Risk Team
- manager@fxrisk.demo - Manager
- admin@fxrisk.demo - Admin

Sur la page login, les boutons Quick Demo permettent une connexion rapide.

10. Commandes utiles

```
npm run dev      Démarrer en développement (port 5173)
npm run build    Construire pour la production
npm run preview  Prévisualiser le build
npm run lint     Vérifier la qualité du code
npm run format   Formater le code avec Prettier
node scripts/generate-guide-debutant.mjs  Régénérer ce PDF
```

11. Glossaire pour débutants

- FX : Foreign Exchange = opérations de change entre devises.
- SWIFT : référence bancaire internationale de virement.
- KRI : Key Risk Indicators = indicateurs clés de risque.
- RLS : Row Level Security = règles qui limitent l'accès aux lignes en base.
- Trigger : fonction SQL exécutée automatiquement à l'insertion ou mise à jour.
- Hook : fonction React réutilisable (useQuery, useSession...).
- Route : une URL associée à une page.
- Migration : script SQL versionné qui modifie la structure de la base.
- SSR : Server Side Rendering = pages générées côté serveur.

12. Conseils pour apprendre

- Lisez README.md pour la documentation complete.
- Explorez src/routes/ une page a la fois.
- Connectez-vous avec chaque role demo pour voir les differences.
- Creez une operation test et suivez son parcours jusqu'a la validation.
- Modifiez un libelle simple, sauvegardez, observez le resultat dans le navigateur.

Document genere pour le projet FX Risk - Plateforme de Gestion des Risques de Change.